Introducción.....2

Formas generales de obtener ayuda en Linux. (Modo Texto).....3

Usuarios y grupos.....4

Comandos para manipular ficheros y directorios.....5

Comando ls 5
Comando cd 5
Comando pwd 5
Comando mkdir 5
Comando mv 6
Comando cp 6
Comando rm 6
Comando chown 7
Comando df.....7
Comando free.....7
Comando alias 7
Comando type 8
Comando stat 8
Comando diff.....8
Comando du 8

Comandos para paginar, visualizar y editar ficheros.....9

Comando cat 9
Editor vi 9
Comandos more y less 10
Comando file 10

Comandos para hacer búsquedas de ficheros y patrones.....11

Comando grep 11
Comando find 11
Comando locate 12

Comandos para filtrar ficheros.....12

Comando sort 12
Comando uniq 13
Comandos tail y head 13
Comando wc 13
Comando cut 14
Comando sed.....14
Comando awk.....16

Comandos para compactar y agrupar ficheros.....18

Comandos gzip y gunzip 18
Comando tar 19

Introducción.

Una vez instalado e inicializado un sistema Linux se dispone de dos vías fundamentales de interacción: una gráfica (si se instaló una interfaz X y se configuró adecuadamente) y una texto conocida como consola o terminal.

Al igual que Unix, Linux ofrece el mecanismo de consolas o terminales virtuales. Este consiste en que a partir de una entrada (el teclado) y con una salida (el monitor) se simulen varias terminales, donde el mismo, o distintos usuarios puedan conectarse indistintamente. De esta forma es posible tener más de una sesión abierta en la misma máquina y trabajar en ellas. Este mecanismo también facilita la característica multiusuario del sistema Linux pues las diferentes conexiones se pueden establecer con diferentes usuarios.

Por defecto, las consolas desde la uno a la seis tienen asociado un programa que permite conectarse al sistema en modo texto, mientras que la siete, si se instaló y activó el ``modo gráfico'', constituye una consola gráfica.

El cambio de una consola a otra se realiza a través de la combinación de teclas Alt y Fx (las teclas de Función), donde x oscila entre 1 y 12. De esta forma se pueden acceder un total de 24 consolas virtuales: para las doce primeras se utiliza el Alt izquierdo y para las otras doce el derecho. Por ejemplo para llegar a la consola 16 se presionarían las teclas Alt derecho y F4. No obstante normalmente solo se puede acceder a las consolas con algún proceso o funcionalidad definida.

Desde una consola gráfica para cambiar a otra tipo texto se debe además presionar la tecla Ctrl, pues las combinaciones Alt + Fx son capturadas e interpretadas por las aplicaciones gráficas de otra forma.

Con la tecla Alt izquierda combinada con los cursores (derecho e izquierdo) se puede además, realizar un movimiento circular entre todas aquellas consolas que tengan un proceso asociado (texto, gráfico, etc.).

Si nos remitimos a una consola texto podremos apreciar que en ella se mostrará, el nombre de la distribución, la versión de la distribución, la versión del kernel y la arquitectura de la máquina. También aparecerá el nombre que se le asignó al sistema en la instalación y la palabra login. Aquí puede entrarse el login de un usuario del sistema (seguramente root, si no se crearon más cuentas durante la instalación). Luego se pedirá el password que al ser entrado no se muestra ningún eco en la pantalla. Si ambos son válidos se establecerá la conexión y se mostrará lo que se conoce como prompt, con forma similar a esta:

```
[root@pinguino /root]$
```

Para abreviar, en lo sucesivo usaremos el carácter $ para hacer referencia al prompt de un usuario común, y # para el prompt del súper usuario root.

Al igual que en Unix, el programa que se ejecuta en Linux cada vez que un usuario se conecta y que le permite interactuar con el sistema se conoce como shell (bash normalmente). Este es capaz de interpretar una gran gama de comandos y sentencias. Constituye a su vez un poderoso lenguaje de Scripting (programación). Muchos programas y comandos son shell scripts, por lo que el conocer este lenguaje permite no sólo entenderlos y ajustarlos a las necesidades particulares sino también, hacer nuevos scripts para automatizar diversas tareas.

Existen muchos tipos de shells orientados a caracteres para Linux.

Ejemplos:

- ash: A shell
- csh: C shell

- tcsh: una extensión al C shell
- ksh: Korn Shell
- bsh: Bourne Shell
- bash: Bourne Again Shell

Algunos shells se originan de otros o toman sus características más avanzadas. De los mencionados anteriormente **bash** es el más empleado en Linux. Este es un producto de la FSF (Free Software Foundation). Es uno de los más desarrollados y está portado para múltiples plataformas. Toma las facilidades de csh y ksh. Ofrece entre otras posibilidades las siguientes:

- Auto completar durante la escritura. Al teclear uno o varios caracteres se puede pulsar TAB con el objetivo de que en caso de que pueda completarse de forma unívoca un comando, nombre de fichero o una variable (en dependencia del contexto), complete de forma automática (se escriba el resto de la palabra). Si existieran varias posibilidades para completar la palabra, se oirá un sonido y volviendo a pulsar TAB se mostrarán en pantalla todas las posibilidades existentes. En caso de existir muchas posibilidades (por defecto más de 100) se pregunta si se desea mostrarlas todas o no.
- Historial de comandos. Esta es una facilidad de muchos otros shells que permite el movimiento a través de los últimos N comandos ejecutados, en la sesión actual o en las anteriores. N por defecto es 1000, pero puede modificarse. Para moverse arriba y abajo se suelen utilizar los cursores.
- Poderosas estructuras de control para realizar scripts. (Procesos por lotes). Se pueden utilizar instrucciones if, for, while, select, case, etc.
- Definición de funciones y alias para comandos. Las funciones permiten definir subrutinas programadas usando el lenguaje de bash y los alias, asociar nombres a llamados a comandos con ciertas opciones y argumentos de forma más nemotécnica o abreviada.

Formas generales de obtener ayuda en Linux. (Modo Texto).

Existen múltiples y variadas formas de obtener ayuda en un sistema Linux. A continuación se describen algunas de ellas:

1) Muchos comandos poseen una opción para mostrar una ayuda breve acerca de su utilización. Esta opción usualmente es **-h**, **--help** ó -?.
2) El comando **man** formatea y despliega un manual bastante amplio acerca de comandos, formatos de ficheros de configuración, llamados al sistema, etc. Los manuales están disponibles y pueden instalarse en múltiples idiomas. Estos se dividen internamente en secciones. Un mismo objetivo puede estar representado en varias secciones. De no especificarse ninguna sección a través del primer argumento del comando se tomará la primera donde aparezca.

   Ejemplo: $ **man chmod**
3) Existe un conjunto de comandos integrados (builtin) al bash que no poseen un manual propio. Para ellos se puede emplear el comando help. Si se usa man sólo se mostrará la lista de los comandos integrados. La ayuda que muestra help es un fragmento del manual de bash ($ **man bash**), el cual es muy amplio y resulta incómoda la búsqueda. Algunos de los comandos integrados al bash y que veremos más adelante son: cd, fg, bg, logout, exit, umask, set, help, source, alias, echo, kill, jobs y export.

   Ejemplo: $ **help logout**

4) El programa **info** despliega información acerca de comandos, en ocasiones más amplia que la que brinda man. Las páginas de info poseen una estructura arbórea (nodos), a través de las cuales se puede navegar con ayuda de comandos especiales.

   Ejemplo: $ **`info ln`**

5) El comando **whatis** realiza una búsqueda en las secciones del man y muestra la descripción abreviada del comando en cada una de las secciones que aparezca. Este comando trabaja con una base de datos que se actualiza periódicamente y se crea con el comando makewhatis.

   Ejemplo: $ **`whatis shadow`**

6) El comando **apropos**, dada una palabra busca en toda la base de datos del whatis desplegando todo lo encontrado. Ejemplo: $ **`apropos passwd`**

7) Muchos programas se empaquetan con su documentación en diversos formatos. Normalmente estas ayudas se agrupan en el directorio **/usr/share/doc**. También existen los **HOWTOs** (Como lograr...) escritos en muchos idiomas y formatos para varios temas disponibles. Algunos se empaquetan como cualquier otro programa o se localizan en Internet,

8) En **Internet** de forma general existen una gran cantidad de grupos de noticias, listas de discusión, sitios Web y FTP sobre Linux. En muchos de ellos incluso se pueden encontrar paginas de ayuda sobre los comandos en castellano, mientras que en nuestra versión de Linux es posible que nos aparezcan en Ingles.

Usuarios y grupos.

Como ya se ha afirmado Linux es un sistema multiusuario, lo cual permite que varios usuarios puedan conectarse y trabajar en él de forma simultánea. Las conexiones como ya se ha visto se pueden realizar a través de varias terminales locales o utilizando servicios de red como el Telnet y el SSH.

Un usuario se caracteriza por su login el cual debe indicar para conectarse al sistema, además de su password o contraseña. Además puede poseer un conjunto de datos adicionales mencionados más adelante.

El usuario con más privilegios en Linux es aquel cuyo login es root. Este es el único con derechos suficientes para crear o eliminar a otros usuarios, además de acceder a todo el sistema de ficheros sin ninguna restricción.

En Linux además existen grupos de usuarios también administrados por root o por un usuario designado por este. Los grupos permiten otorgar los mismos privilegios a un conjunto de usuarios.

Siempre que se añada un usuario al sistema se creará un grupo con su mismo nombre, llamado grupo primario. Durante la creación o posteriormente, se podrá incorporar el usuario a otros grupos secundarios.

Tanto los usuarios como los grupos se identifican por el sistema a través de un identificador (ID) numérico. El usuario root siempre tiene el ID cero. Cada usuario cuando se conecta al sistema posee un identificador de usuario asociado (uid) y un identificador de grupo (gid).

Para comprobar el uid de nuestro usuario, y el gid del grupo principal al que pertenece, podéis ejecutar la orden **id**.

Al añadir un usuario también se creará un directorio base para el mismo con el nombre de su login. Este directorio se coloca por defecto en el directorio /home excepto para root, cuyo directorio base es /root.

La información asociada a los usuarios en un sistema Linux se guarda en el fichero /etc/passwd y las contraseñas y datos afines en /etc/shadow. Por su parte la información de los grupos, sus miembros y passwords están en /etc/group y /etc/gshadow respectivamente.

Comandos para manipular ficheros y directorios.

Comando ls

El comando ls permite listar el contenido de un directorio.

Sintaxis: ls [opciones] [directorio | fichero]

Algunas opciones:

- -l : muestra la salida en formato largo.
- -R: lista recursivamente un directorio.
- -a : lista además los ficheros ocultos (sus nombres comienzan con punto).
- -h : muestra el tamaño de los ficheros en forma más legible (Ej.: 16M, 4k, etc.)
- -i : muestra el identificador del i-nodo asociado a cada elemento.

Ejemplos:

```
$ ls -hl /etc
$ ls -R /usr
$ ls -al
$ ls -ali ..
```

Un grupo de opciones que se suele utilizar bastante es lia (ls –lia)

Comando cd

El comando cd se utiliza para cambiar el directorio actual.

Sintaxis: cd [directorio]

Ejemplos:

```
$ cd /tmp
$ cd # cambia al directorio home del usuario.
$ cd ~ # cambia al directorio home del usuario.
```

Comando pwd

El comando pwd indica el camino absoluto del directorio en el cual nos encontramos actualmente.

Ejemplo:

```
$ pwd   # nos devuelve algo como /home/pepe/backup/pruebas
```

Comando mkdir

El comando mkdir se utiliza para crear directorios.

Ejemplos:

```
$ mkdir bin
$ mkdir /bin
```

```
$ mkdir -p docs/linuxdocs/howtos/pdf # se crean los directorios intermedios si es necesario
```

Comando mv

El comando mv mueve un fichero hacia otro, o varios ficheros hacia un directorio. Este permite a su vez renombrar ficheros o directorios.

Sintaxis: mv [opciones] <fuente> <destino>

mv [opciones] <ficheros> <directorio>

Algunas opciones:

- -i : ejecuta el comando de forma interactiva, o sea, pregunta ante de sobrescribir el destino si existiera.
- -u : actualiza (upgrade) el destino con el fuente solo si este es más reciente.

Ejemplos:

```
$ mv mail.cf mail.cf.old      # renombra un fichero
$ mv -i *.txt /tmp            # mueve ficheros terminados en .txt al directorio /tmp
$ mv -u program.c src/   # actualiza el fichero destino si es menos reciente que el fuente
```

Comando cp

El comando cp permite copiar un fichero en otro, o varios ficheros en un directorio.

Sintaxis:

cp [opciones] <fuente> <destino>

cp [opciones] <ficheros> <directorio>

Algunas opciones:

- -R : copia recursivamente un directorio.
- -i : utiliza una forma interactiva (pregunta antes de sobrescribir el destino).
- -l : hace enlaces fuertes a los ficheros fuentes en lugar de copiarlos.

Ejemplos:

```
$ cp /etc/passwd .    # copia un fichero en el directorio actual
$ cp -i /usr/bin/*sh /tmp   # copia interactivamente los ficheros terminados en sh en un directorio llamado /tmp
```

Comando rm

El comando rm se utiliza para borrar (desenlazar) ficheros

Sintaxis: rm [opciones] <ficheros | directorios>

Algunas opciones:

- -r : borra recursivamente un directorio.
- -f : borra forzosamente en caso de que no se tenga permiso de escritura en forma directa.
- -i : ejecuta el comando de forma interactiva.

Ejemplos:

```
$ rm prueba
$ rm -i bin/*
$ rm -rf temp/
```

Comando chown

El comando chown se utiliza para cambiar el dueño y el grupo de un fichero. Existe también el comando chgrp que se emplea de forma similar pero para cambiar el grupo solamente. El dueño de un fichero solo lo puede cambiar el usuario root mientras que el grupo además de root, lo puede cambiar el propio dueño, siempre que pertenezca al nuevo grupo.

El propietario y el grupo de un fichero se puede comprobar con un ls -l

Sintaxis:

chown [opciones] <dueño>[.grupo] <ficheros>

chown [opciones] <grupo> <ficheros>

Opción:

- -R en los directorios cambia el dueño y/o el grupo recursivamente.

Ejemplos:

```
# chown pepe.pepe tesis/ # cambia el propietario de tesis a pepe y el grupo de tesis al grupo pepe
# chown -R root /tmp/oculto # cambia todos los ficheros que esten en el directorio oculto y coloca como propietario al usuario root
# chgrp ftp /usr/ftp
```

Comando df

El comando nos informa del uso de disco en los distintos volúmenes. Asi podemos saber cuanto espacio tenemos en cada volumen, y cuanto esta usado y cuanto libre.

```
# df -h
```

Comando free

El comando free nos informa del consumo de memoria del sistema, tanto en memoria real como en memoria virtual (swap).

```
# free
```

Comando alias

El comando alias permite asignarle otros nombres a los comandos. De esta forma se pueden abreviar o llamarlos de forma más nemotécnica.

Sintaxis: alias [nombre[=valor]...]

Sin argumentos muestra todos los alias definidos por el usuario actual. Para deshabilitar un alias se emplea el comando unalias.

Ejemplos:

$ alias l='ls -l --color'

$ alias l

Comando type

El comando type indica como interpreta el bash una orden En caso de ser un comando mostraría el camino (path) de este, si fuera un alias, cual es el comando original y para una función mostraría su código. Una misma palabra puede constituir un comando, un alias y una función. Bash interpreta con mayor prioridad la función, luego el alias y por último, el comando.

Sintaxis: type [-all] [-path|-type] [palabras]

Opciones:

-all : muestra todas las posibilidades de interpretación.

-path : muestra el camino completo al fichero que representa al comando, en caso de serlo.

-type : muestra la forma de interpretación asumida: alias, keyword, function, builtin y file.

Ejemplos:

```
$ type l                    l is aliased to `ls -l --color'
$ type passwd               passwd is /usr/bin/passwd
# type -all rm              rm is aliased to `rm -i'
                            rm is /bin/rm
```

Comando stat

El comando stat muestra las características de un fichero. Por ejemplo: su nombre, permisos, tamaño en bytes, número del i-nodo que lo representa, las fechas de modificación y acceso, el tipo, el dueño, el grupo, etc.

Ejemplos**:** **$ stat /etc/shadow**

Comando diff

El comando diff nos permite comparar ficheros de texto línea a línea, y nos indica en caso de que no sean iguales en que líneas cambian. Dado que en Linux se realizan muchas configuraciones del sistema modificando ficheros de texto, es una orden interesante para saber si algo se ha modificado.

```
# diff fichero1 fichero2
```

Comando du

El comando du permite conocer la longitud (expresada en kilobytes por defecto) de una jerarquía de ficheros a partir de un directorio.

Sintaxis: du [opciones] [ficheros | directorios]

Algunas opciones:

-h : (human readable view) imprime las unidades de la forma más representativa (Ej. M para megabytes).

-s : sumariza el tamaño de cada fichero/directorio sin profundizar recursivamente en estos últimos.

-c : produce un total cuando se utilizan varios argumentos.

Ejemplos:

```
$ du -h *
$ du -hsc /usr /home
```

Comandos para paginar, visualizar y editar ficheros

Comando cat

El comando cat concatena (conCATenate) ficheros y los imprime en la salida estándar. Si no se le pasa ningún argumento lee de la entrada estándar. Existe también zcat que hace lo mismo pero con ficheros comprimidos. Si solo se da el origen a cat, utiliza como salida la pantalla. Es decir, cat hola muestra por pantalla el fichero hola. Si solo se da la salida a cat (cat > fichero) utiliza como entrada el teclado.

Ejemplo:

```
# cat /etc/passwd /etc/shadow
$ cat > fichero
$ cat < origen > destino
```

Editor vi

El editor vi es el editor estándar de Unix y esta orientado a comandos. Existe una versión conocida como vim (Vi IMproved) muy poderosa que permite la edición de múltiples ficheros, edición automatica para varios lenguajes de programación, ayuda en línea, selección visual, varios niveles de deshacer, etc. Para algunos usuarios, vi resulta incómodo pues para utilizar todas sus potencialidades es necesario conocer muchas combinaciones de teclas, pero si se llega a dominar resulta muy funcional. Su principal virtud es que encontraremos vi en prácticamente cualquier versión de Unix que usemos, cosa que no se puede decir de otros editores (joe, pico, edit, gedit, nano, emacs, etc.).

Básicamente vi posee dos modos de interacción: el de inserción (edición) y el de comandos. Para pasar al modo comando se pulsa **Esc** y para pasar al de inserción se pulsa **i**.

Algunos comandos útiles en vi (pulsando ESC para pasar al modo de comandos, que veremos en la ultima línea de la pantalla).

- dd - borra la línea actual.
- D - borra desde la posición actual hasta el final de la línea.
- dG - borra hasta el final del fichero.
- u - deshace el último comando.
- :q - sale del editor (si se han hecho modificaciones y no se ha salvado se genera un error).
- :q! - sale sin salvar.
- :w - salva.
- :wq - salva y sale.
- :x - salva y sale.
- <n><comando> - ejecuta el comando c n veces.

Comandos more y less

Los comandos more y less paginan (dividen en páginas) uno o varios ficheros y los muestran en la terminal (pantalla). De no indicárseles un fichero, paginan la entrada estándar (que se manda mediante una tuberia).

Se diferencian en las facilidades que brindan. Por ejemplo more es más restrictivo en cuanto al movimiento dentro del texto, mientras que less no limita este aspecto pues acepta el empleo de todas las teclas de movimiento tradicionales. Cuando se alcanza el final del último fichero a paginar, more termina automáticamente, no así less. También more muestra sucesivamente el porcentaje del fichero visto hasta el momento. Tanto less como more proveen una serie de comandos para moverse con facilidad dentro del texto paginado.

Ejemplos:

```
$ less /etc/passwd
$ more /etc/passwd
$ cat fichero | less
```

Algunas teclas que podemos usar mientras usamos estos programas son:

- q - permite interrumpir el proceso y salir.
- /p - realiza búsquedas del patrón p dentro del texto. Para repetir la búsqueda del mismo patrón sólo es necesario escribir /.
- [n]b - en more permite regresar n páginas (por defecto n es 1).
- [n]f - en more se adelantan n páginas y en less, n líneas.

El man, para dar formato a su salida, utiliza por defecto el paginador less. Existen además los comando zless y zmore que permiten paginar con less y more respectivamente, a los ficheros comprimidos sin necesidad de descomprimirlos previamente.

Comando file

El comando file determina con cierto grado de precisión el tipo de un fichero que se le pasa como argumento.

Ejemplos:

```
$ file /etc/passwd
$ file /usr/sbin/adduser
$ file /usr/sbin/useradd
$ file /etc
```

Comandos para hacer búsquedas de ficheros y patrones

Comando grep

El comando grep (Globally Regular Expressions Pattern) busca patrones en ficheros. Por defecto devuelve todas las líneas que contienen un patrón (cadena de texto) determinado en uno o varios ficheros. Utilizando las opciones se puede variar mucho este comportamiento. Si no se le pasa ningún fichero como argumento hace la búsqueda en su entrada estándar.

Sintaxis: grep [opciones] <patrón> [ficheros]

Algunas opciones:

- -c devuelve sólo la cantidad de líneas que contienen al patrón.
- -i ignora las diferencias entre mayúsculas y minúsculas.
- -H imprime además de las líneas, el nombre del fichero donde se encontró el patrón. Es así por defecto cuando se hace la búsqueda en más de un fichero.
- -l cuando son múltiples ficheros sólo muestra los nombres de aquellos donde se encontró al patrón y no las líneas correspondientes.
- -v devuelve las líneas que no contienen el patrón.
- -r busca en un directorio de forma recursiva.
- -n imprime el número de cada línea que contiene al patrón.

Ejemplos:

```
$ grep linux /usr/share/doc
$ grep root  /etc/passwd
# grep -n error /var/log/messages
$ grep -i pepe /etc/passwd
$ grep -c root /etc/group
$ grep -l -r -i img /var/www/html/
$ ls –lia | grep "carta roja"
```

Comando find

El comando find es uno de los más poderosos en un sistema Linux. Permite buscar de forma recursiva en un directorio a todos los ficheros que cumplan ciertas condiciones. Las condiciones pueden estar relacionadas con el nombre de los ficheros, el tamaño, los permisos, el tipo, las fechas de acceso y modificación, etc.

Sintaxis: find [camino] [opciones]

Algunas opciones:

- **-name <expresión>** permite especificar patrones para los nombres de los ficheros a buscar.
- **-iname <expresión>** permite especificar patrones para los nombres de los ficheros a buscar sin tener en cuenta mayúsculas y minúsculas.

- **-type <tipo>** permite indicar el tipo de fichero a buscar. Este puede ser d para directorios, f para ficheros regulares, l para enlaces simbólicos, b para dispositivos de bloque, c para dispositivos de carácter, p para tuberías y s para sockets.
- **-size +/-<n>** permite indicar el tamaño máximo y/o mínimo de los ficheros a buscar. Por defecto el tamaño se expresa en bloques de 512 bytes, pero si se precede este por un carácter c se referirá a bytes, k a kilobytes, w a palabras de dos bytes y b a bloques.
- **-perm [+|-]<modo>** permite referirse a aquellos ficheros cuyos permisos sean exactamente modo, incluya todos los de modo (signo -) o incluya alguno de los de <modo> (signo +). El valor de <modo> se expresa en forma numérica.
- **-exec <comando> ;** permite definir un comando a ejecutarse para cada resultado de la búsqueda. La cadena {} se sustituye por el nombre de los ficheros encontrados. El caracter ; permite indicar la finalización del comando. (Tanto {} como ; tienen que ir entre comillas o entre contrabarras para evitar que sea sustituido por el shell).

Ejemplos:

```
$ find /etc -name '*.conf'   # busca en /etc todos los ficheros con extensión conf

$ find / -size +10240k -size -20480k     # busca los ficheros cuyo tamaño esté entre 10M y 20M

$ find -perm +1000 -type d       # busca los directorios que posean el permiso t

# find / -name core -exec rm -i “{}” “;”      #   busca   todos   los ficheros que se nombren core y los borra interactivamente. Los signos “” se utilizan para proteger de la interpretación del shell
```

Comando locate

El comando locate busca en una base de datos, actualizada periódicamente, todos los paths en la jerarquía de ficheros que contengan una cadena determinada. Para crear esta base de datos o actualizarla se debe invocar por root el comando updatedb (o locate -u).

En las distribuciones actuales esta tarea se hace de forma automática, aunque el usuario root normalmente tiene que ejecutarlo al menos una vez para crear la base de datos).

Ejemplo:

```
$ locate passwd
```

Comandos para filtrar ficheros

Comando sort

El comando sort ordena las líneas de un fichero mostrándolas por la salida estándar. De no especificarse un fichero toma la entrada estándar.

Sintaxis: sort [opciones] [fichero]

Algunas opciones:

-r : ordena al revés.

-f : trata las mayúsculas y minúsculas por igual.

-g : ordena de forma numérica, de modo que no es necesario que los números se rellenen con ceros por la izquierda.

Ejemplo: **$ sort -f /etc/passwd**

Como ejercicio, cread un archivo llamado lista-desordenada con el vi y meter dentro 5 nombres desordenados. Comprobad como con la orden sort lista-desordenada > lista-ordenada creamos un fichero llamado lista-ordenada y que contiene la lista ordenada.

Comando uniq

El comando uniq elimina las líneas repetidas de un fichero ordenado, imprimiéndolo por la salida estándar o en otro fichero argumento. De no especificarse un fichero toma la entrada estándar.

Sintaxis: uniq [opciones] [fichero] [salida]

Algunas opciones:

-c : utiliza como prefijo en cada línea su número de ocurrencias.

-d : solo imprime las líneas duplicadas.

Ejemplo**: $ uniq -d lista.txt**

Comandos tail y head

Los comandos tail y head muestran respectivamente el final y el comienzo (10 líneas por defecto) de uno o varios ficheros. De no especificarse al menos un fichero toman la entrada estándar.

Sintaxis:

tail [opciones] [ficheros]

head [opciones] [ficheros]

Algunas opciones:

- -f para el caso de tail se ejecuta de forma sostenida o sea se continúa visualizando el final del fichero hasta que se interrumpa el proceso (Ctrl-c).
- -q no coloca los encabezamiento con el nombre de los ficheros cuando se indican varios (quiet).
- -<n> imprime las n últimas (primeras) líneas en lugar de las diez establecidas por defecto.

Ejemplos:

```
# tail -f /var/log/messages
# tail -20 /var/log/secure
# head -50 /var/spool/mail/pepe
# head -2 -q /etc/*.conf
```

Comando wc

El comando wc imprime el número de líneas, palabras y bytes de uno o varios ficheros. Si son varios ficheros hace también un resumen de los totales. De no especificarse un fichero toma la entrada estándar.

Sintaxis: wc [opciones] [ficheros]

Algunas opciones:

- -l sólo cuenta líneas.
- -c sólo cuenta bytes.
- -w sólo cuenta palabras.

Ejemplos:

```
$ wc -l /etc/passwd
$ wc -w /doc/dicciorario.txt
```

Comando cut

El comando cut nos permite cortar una línea de texto, para obtener un subconjunto en lugar de la línea completa. Podemos cortar por número de caracteres, por campos, etc.

Sintaxis: cut [opciones] [ficheros]

Algunas opciones:

- -c N-M corta desde el carácter número N hasta el carácter número M.
- -c N- corta desde el carácter número N hasta el final
- -c –N corta desde el principio hasta el carácter número N
- -c N,M corta el carácter número N y el carácter número M
- -d”:” –f1 separa la línea en campos divididos por el carácter : y nos muestra sólo el primer campo
- -d”-“ –f3 separa la linea en campos divididos por el carácter – y nos muestra sólo el 3 campo.

Ejemplos:

```
$ cut –c 3-9 /etc/passwd
$ cut –c d”:” –f4 /etc/passwd
```

Comando sed

Sed realmente es un mini lenguaje de programación capaz de hacer sustituciones realmente complejas de cadenas de texto.

Sintaxis: sed –n –e”comandos” –f archivo [opciones] [ficheros]

Algunas opciones:

- -n no produce la salida estándar.
- -e”comandos” indica que comandos va a ejecutar sed
- -f si queremos que los comandos se tomen de un archivo

Los comandos de sed, se usan de la siguiente manera:

La estructura general es: **[inicio[,fin]] función [argumentos]**

Donde inicio y fin hacen referencia a las líneas que se van a tratar, función hace referencia a que acción vamos a realizar, y argumentos indica como se va a aplicar la acción.

Si queremos poner varios comandos en varias líneas, al final de cada línea se debe añadir la contrabarra (\) antes de pulsar intro para que nos deje seguir escribiendo.

Los comandos más habituales en sed son:

a\ Añade al final de la línea

c\ Cambia el contenido del patrón de texto

d Borra las líneas indicadas

s Realiza sustituciones

g Realiza sustituciones generales

i\ Inserta líneas

p Imprime líneas

q Abandona sed

r arch Lee el archivo y lo añade a la salida

w arch Copia las líneas en un archivo

= Da el número de línea

/^tex/ Realiza un grep simple buscando tex

Ejemplos:

$sed "3d" canciones Produce una salida donde se borra la 3ª línea de canciones

$sed "a\Hola" canciones Produce una salida donde después de cada línea se escribe hola en una línea nueva.

$sed "i\Hola" canciones Produce una salida donde antes de cada línea se escribe Hola en una nueva línea.

$sed –n 2p canciones Solo nos muestra la 2ª línea de canciones.

$sed –n 3,9p canciones Solo nos muestra las líneas 3 a la 9 de canciones

$sed –n '3,$p' canciones Solo nos muestra de la línea 3 al final. (fijaos como hay que usar aquí comillas simples para evitar que $p crea que es una variable).

$sed –n /^ho/ canciones Solo nos muestra las líneas que comiencen por ho

$sed –n '/^t/,$p' canciones Nos muestra desde la primera línea que comience por t hasta la ultima línea.

$sed 's/antes/después/g' canciones Sustituye todos los "antes" por "después" en el fichero canciones.

$sed 's/antes/después/' canciones Sustituye el primer antes por después en cada línea….

$sed '1,6s/antes/después/g' canciones Sustituye todos los antes por despues, pero solo en las líneas 1 a 6.

$sed '/^en/s/jose/pepe/g' canciones Sustituye todos los jose por pepe, pero solo en aquellas líneas donde se encuentre la cadena "en"

$sed 'y/[123]/[456]/' canciones Sustituye los 1 por 4, los 2 por 5, y los 3 por 6

$sed 's/juana/#&/' profesores Sustituye Juana, por la #Juana (& indicael texto buscado) pero poniendole un # delante, es decir, que la comenta.

$sed –n '/pirata/p' poema Solo nos muestra las líneas donde aparezca pirata en poema.

Comando awk

Awk es un verdadero lenguaje de programación, especializado en análisis y procesamiento de patrones de texto. Viene a ser como un grep mezclado con sed.

Sintaxis: awk [-f archivo_programa] [-Fc] ['programa'] [variable=valor ...] [archivo]

donde:

- archivo_programa : especifica el archivo fuente del programa a aplicar a archivo.
- c : especifica el carácter delimitador de campos. Por defecto se considera el espacio en blanco.
- programa: especifica el conjunto de patrones e instrucciones a ejecutar con archivo. La diferencia con archivo_programa, es que en este caso hay que especificar los patrones e instrucciones en la línea de comandos. Para evitar interferencias con la shell deben ir encerradas entre comillas simples (').
- variable=valor : se utiliza para establecer los valores que tomarán las variables que utilice el programa.
- archivo : archivo que será procesado por awk. Si se especifica "-" se interpretará como la entrada estándar.

Como ya hemos indicado, awk es un verdadero lenguaje de programación, es decir, que cuenta con sus instrucciones secuenciales, alternativas, reiterativas, etc.

Awk se basa en el uso de variables, donde $0 se refiere a una línea completa, $1 al primer campo, $2 al segundo campo, etc.

Asi, por ejemplo, la siguiente instrucción nos muestro los nombres de usuarios del fichero passwd, que como ya sabemos cuenta con campos separados por el símbolo dos puntos.

```
awk –F:  `{ print $1 } ’  /etc/passwd
```

En el ejemplo anterior, podriamos haber creado un fichero, por ejemplo primercampo.awk que tuviera como contenido { print $1 } y entonces la linea podria haber quedado como:

```
Awk –F:  -f primercampo.awk  /etc/passwd
```

Si no indicamos el campo delimitador se usa por defecto el espacio en blanco. Cread un fichero texto.txt de texto de varias líneas, con varias palabras en cada linea y ejecutad la siguiente instrucción.

```
Cat texto.txt | awk ‘{ print “1ª y 2ª palabras: ” , $1, $2,  “ y fin” }’
```

Awk dispone de varias palabras reservadas que podemos usar para mostrar información sobre la entrada. Asi tenemos por ejemplo:

- FS (Field separator): contiene el carácter que indica a awk en qué punto del registro acaba un campo y empieza el siguiente. Por omisión es un espacio. Se puede indicar un carácter alternativo mediante una instrucción de asignación como FS = “/”. Si se deja vacío, cada lectura se realiza dejando un carácter en cada campo.
- NF (Number of fields): contiene el número total de campos que contiene el registro que se está leyendo en cada momento.
- RS (Record separator): contiene el carácter que indica a awk en qué punto del archivo acaba un registro y empieza el siguiente. Es “\n” por omisión.
- NR (Number of record): contiene el número de orden del registro que se está procesando en cada momento.

- OFS (Output FS): contiene el separador de campos para la salida generada por awk. La instrucción print inserta en la salida un carácter de separación cada vez que aparece una coma en el código. Por ejemplo:

Asi, una línea de orden como la siguiente:

```
awk '{print NR, $0}'
```

nos muestra todas las líneas del fichero passwd ($0) pero con el numero de linea (NR) delante

```
awk '{print "Linea : ", NR, " = ", $0}'
```

awk también permite realizar condicones sobre las líneas, por ejemplo:

```
cat /etc/passwd | awk –F: '$1 == "joancadi" {print "Esta linea tiene en su primer campo la palabra joancadi ->", $0, " y es la linea número ", NR}',
```

Vemos como al poner '$1 == "joancadi" en realidad estamos preguntado si el primer campo es la cadena joancadi, el {print solo se ejecutará si esa condición es verdadera.

Usemos el awk realmente como un lenguaje de programación…. Cread un fichero con nombre sumatorio.awk que tenga las siguientes líneas:

```
BEGIN { total = 0 }
      { total += $1 }
END { print "El total es", total }
```

Cread ahora un fichero con varios numero en varias filas, con nombre numeros.txt, y ejecutad:

```
Cat numeros.txt | awk –f sumatorio.awk
```

Podéis comprobar en este ejemplo como podemos definir variables (total), realizar operaciones aritméticas sobre ellas (+=), realizar bucles, etc. El bucle BEGIN – END se ejecuta por cada línea del fichero que trate el awk.

También tenemos la estructura If – Else….

```
{ print "Procesando linea :", NR;
 if( $1 > 10 )
   { print $1 "es mayor que 10" }
 else
   { print $1 "es menor que 10" }
}
```

No podía faltar el for, claro….

```
{  for( i = 0; i < 10; i ++ )
 {
  j = 2 * i;   print i, j;
 }
}
```

Sin olvidar el Do – While, claro esta…

```
BEGIN { i = 0; do {print i; i ++} while ( i < 10 ) }
```

Tenemos funciones en awk, muchas funciones:

```
awk 'BEGIN { print “El seno de 30 grados es”, sin( 3.141592653589 / 6 ); }' /dev/null
```

Algunas de las funciones más interesantes de awk, son:

length( ) Longitud del parámetro en bytes	rand( ) Número al azar entre cero y uno
srand( ) Inicia la semilla de generación al azar	int( ) Devuelve el parámetro convertido en un entero
substr(s,m,n) Devuelve la subcadena de s en m con longitud N	index(s,t) Posición de s donde aparece t, o cero si no está.
match(s,r) Posición de s en que se cumple la expresión r.	split(s,a,fs) Devuelve s en elementos separados por fs
sub(r,t,s) Cambia en s la cadena t por r. Es $0 por omisión	gsub(r,t,s) Igual, pero cambia todas las t en la cadena
sprintf(f,e,e...) Devuelve cadena de formato f con las “e”	system(cmd) Ejecuta cmd y devuelve el código de retorno
tolower(s) Devuelve s convertida en minúsculas	toupper(s) Devuelve s convertida en mayúsculas
getline Fuerza una lectura de fichero	

Comandos para compactar y agrupar ficheros

Comandos gzip y gunzip

Los comandos gzip y gunzip permiten compactar y descompactar (comprimir y descomprimir) respectivamente uno o varios ficheros.

Sintaxis:

gzip [opciones] <ficheros/directorio>

gunzip [opciones] <ficheros/directorio>

Algunas opciones:

- -r : dado un directorio comprime todos los ficheros presentes en él recursivamente.
- -1 a -9 : especifica el grado de la compresión (-1 menor y más rápida -9 mayor y más lenta).
- -S <sufijo> : permite especificar un sufijo o extensión para el fichero resultado (por defecto es gz).

Ejemplos:

```
$ gzip -9 * # Comprime todos los ficheros del directorio actual (su extensión cambia a .gz)
$ gunzip big-file.gz # descomprime el fichero big-file.gz
```

Todo lo compactado con gzip se puede descompactar con el Winzip de los sistemas Windows. También existen los pares de comandos zip y unzip (compatibles en ambos sentidos con Winzip), y compress y uncompress.

Comando tar

El comando tar (Tape Archiver) es una herramienta para agrupar varios ficheros aislados o el contenido de un directorio en otro fichero o dispositivo especial. El comando tar no comprime o compacta absolutamente nada, se limita a agrupar varios ficheros en uno solo, sin comprimirlos. Existe una opción (-z) que automáticamente ejecuta un gzip o un gunzip sobre el fichero agrupado.

Sintaxis: tar [opciones] <fuentes>

Algunas opciones:

- -c permite crear (tarear), es decir, agrupar ficheros en uno solo.
- -x permite extraer (destarear), es decir, desagrupar ficheros.
- -v activa el modo debug, donde se ven todos los mensajes.
- -f <fichero> agrupa o desagrupa en o hacia un fichero y no utilizando la salida o entrada estándar como es por defecto. (Ojo, esta opción la usaremos siempre).
- -z compacta o descompacta el fichero resultante una vez agrupado o desagrupado con gzip y gunzip respectivamente.
- -t lista el contenido de un fichero resultado de un agrupamiento.
- -M agrupa en volúmenes.

El comando tar conserva la estructura jerárquica original de lo agrupado excluyendo el carácter / que representa a la raíz. Algunas opciones se pueden emplear sin el carácter -, siempre y cuando no haya ambigüedades entre ellas o con los argumentos.

Ejemplos:

$ tar cvzf grande * # crea un fichero grande donde estarán agrupados todos los ficheros del directorio actual y que además estará comprimido.

$ tar xvzf grande # desagrupa en el directorio actual el fichero grande y ademas lo descomprime.

 # tar cf uconf.tar /etc/passwd /etc/shadow /etc/groups # agrupa en el fichero uconf.tar los ficheros passwd shadow y groups

 # tar tf uconf.tar # muestra los ficheros agrupados en uconf.tar

 # tar xf uconf.tar # desagrupa el fichero uconf.tar

$ tar cMf /dev/fd0 /tmp/etc.tgz

$ tar xMf /dev/fd0

 # dmesg